

University Of Essex
Bachelor of Science (Honours) in Computer Science
(Top-up)

CE301 Individual Capstone Project Final report

Project Title: BiteSpace

Student: Marc Ko Da Wei

Student Number: PRID75600

Supervisor: Professor Ziyuan

Second Assessor: Professor Hai Kian

Acknowledgements

Introduction

I wish to convey my sincere appreciation to all those who played a role in the successful completion of this report.

Specific Thanks

Professor Ziyuan: For their unwavering guidance and support throughout this project.

Professor Hai Kian: For their invaluable ideas, insightful feedback, and thoughtful advice, which greatly enhanced the quality of this work.

Professor Kang Leng: For their guidance and the enriching lectures that provided a strong foundation for this project.

Mr Kevin Lo: For his consistent help and unwavering support throughout the course.

General Thanks

I would also like to express my heartfelt appreciation to my peers for their unwavering encouragement and constructive feedback, which motivated me to strive for excellence throughout this journey. Additionally, I am deeply grateful to University of Essex for providing the essential resources, facilities, and a conducive environment that were instrumental in the successful completion of this work." conducive environment that were instrumental in the successful completion of this work.

Abstract

The app is an innovative platform designed to blend social media with food reviews, creating a dedicated space for food enthusiasts to connect, share, and discover dining experiences. Users can explore a curated list of restaurants, post reviews, and check in at their favorite locations, while sharing engaging posts through multimedia content. By leveraging a tiered rewards system, the app incentivizes users to actively contribute their experiences, offering discounts and exclusive perks for their engagement. This unique combination enhances visibility for restaurants and fosters a vibrant community of food lovers, ultimately transforming the way users interact with culinary experiences.

Table of Contents

Title Page	1
Acknowledgements	2
Abstract	3
Table of Contents	4
List of Symbols	8
Chapter 1: Literature Review	9
1.1 Background	9
1.2 Motivation	11
1.3 Project Aim	12
1.4 Project Objectives	13
Chapter 2: Technical Documentation	14
2.1 Project Scope	14
2.1.1 User Features	15
2.1.2 Restaurant Features	16
2.1.3 Technical Requirements	17
2.1.4 Cloud Storage for Image upload/retrieval	17
2.1.5 Search Functionality and Autofill	18
2.2 Application Development	18
2.2.1 Overview of Application Structure	19
2.2.2 Front End Development	20
2.2.2.1 Pros and Cons of Expo	21
2.2.2.2 Pros and Cons of Typescript	22

2.2.3 Backend Development	23
2.2.3.1 Setup	23
2.2.3.2 Models	25
2.2.3.2.1 Models Overview	25
2.2.3.2.2 Relationships and Interactions	26
2.2.3.3 Routes	26
2.2.3.3.1 Test routes	26
2.2.3.3.2 GET routes	27
2.2.3.3.3 POST routes	29
2.2.3.4 Elasticsearch	31
2.2.4 SQL	33
2.2.4.1 Communication Diagram	34
2.2.5 Supabase	34
2.2.6 Use Case Scenario & Use Case Diagrams	34
2.2.6.1 UC-1 Login/Register	35
2.2.6.2 UC-2 Logged in screen	37
2.2.6.3 UC-3 Home Tab	39
2.2.6.4 UC-4 Promotions Tab	40
2.2.6.5 UC-5 Post Tab	41
2.2.6.6 UC-6 Newsfeed Tab	42
2.2.6.7 UC-7 Profile Tab	44
2.3 Figma Designs & Test cases	45
2.3.1 Landing Screen	45
2.3.2 Registration	45

2.3.3 Login	49
2.3.4 Tab layout	51
2.3.5 Home Screen	51
2.3.6 Review Screen	56
2.3.7 Post Screen	57
2.3.8 Newsfeed Screen	59
2.3.8.1 Search for friends	61
2.3.8.2 Routing from Newsfeed to client profiles	61
2.3.9 Other Profiles	61
2.3.10 Profile tab	63
2.3.11 Quest Screen	64
Chapter 3: Project Planning	66
3.1 Jira	66
3.1.1 Epics	67
3.1.2 Stories	68
3.1.3 Retrospective	69
3.1.4 Did the Methodology Prove Suitable?	70
3.1.5 Risk Management	71
Chapter 4: Conclusions	73
4.1 Achievements	74
4.2 Bugs	74
4.3 Future Work	74
4.3.1 Features	75
4.3.2 Scalability	75

4.4 Learning Outcomes	76
References	78

List of Symbols

E2E: End to End

ICPC: Individual Capstone Project Challenge

API: Application Programming Interface

OTA: Over-the-Air

RDBMS: relational database management system

ACID : Atomicity, Consistency, Isolation, Durability

UC: Use case

Chapter 1. Literature Review

1.1 Background

The modern dining experience has transcended beyond the simple act of eating. It has become a social and cultural activity deeply intertwined with technology and social media. Food enthusiasts and consumers increasingly rely on online platforms to discover restaurants, share dining experiences, and connect with like-minded individuals.

As social media becomes an integral part of our daily routines, it's evident that these platforms significantly influence the way we eat. Over 90% of adolescents have at least one social media account, and their presence on social media continues to grow.[1] According to a research, social media significantly influences food choices and taste preferences (Nelson & Fleming, 2019; Onorati & Giardullo, 2020).[2] As such, a social media integration with advertising features apart from just regular ads holds immense untapped potential which could be leveraged by restaurants as a stepping stone to success.

Despite the presence of various review sites and social media apps, there remains a gap in the market for a dedicated platform that merges these functionalities, offering a seamless, community-driven experience for food lovers. Currently, consumers searching for restaurants will have all factors such as past experience of previous consumers, e-word of mouth through blogs, articles, comments and review, and advertising of restaurants on various social media platforms.[3] The problem where there is no solid platform for targeted advertising persists.

In today's fast-paced world, finding trustworthy recommendations can be a challenge. Many existing platforms prioritize sponsored content, making it difficult for users to identify genuine reviews and hidden gems. Additionally,

these platforms often lack engaging social features, reducing their ability to foster a sense of community among users.

Currently in the market, there is a slightly similar platform called Yelp. In a recent study, Anderson and Magruder show that higher Yelp scores help restaurants sell tables 19 percent more frequently during peak periods. This impact is strongest for restaurants for which information is the scarcest. Restaurants that are not accredited by experts sell out 27 percent more often if they earn an extra star on Yelp. [4] Yelp, however, is multifaceted with many broader categories apart from food.

For restaurants, existing platforms often fail to provide a dedicated space for targeted advertisement and engagement with their ideal audience. Marketing efforts are frequently lost in the clutter of general social media or review sites, where the focus is not exclusively on food. Restaurants lack a niche community to showcase their offerings effectively to an audience genuinely interested in dining experiences. By having a platform dedicated to food enthusiasts, restaurants can directly connect with a saturated, highly engaged community, enabling them to promote their dishes, events, and services to a receptive audience while building stronger brand loyalty and visibility.

This need for a dedicated space aligns with the concept of niche marketing, defined as targeted marketing intended at a small, specialized market with few target groups and rivals. Niche marketing can be especially beneficial for small or specialized businesses like restaurants, as it allows them to concentrate their efforts on a specific audience.[5] Unlike mass marketing strategies that aim to reach the broadest possible consumer base, niche marketing enables restaurants to tailor their messaging, promotions, and engagement strategies to a community that shares a deep interest in food and dining experiences. As a result, restaurants can increase their relevance and visibility within a segment that is highly likely to convert into loyal customers.

1.2 Motivation

Empowering Users with a Focused Community:

Unlike traditional social media platforms, this app is dedicated to food-related content, providing users with a curated experience that celebrates culinary exploration. Users are rewarded for sharing authentic reviews, photos, and videos, encouraging them to actively contribute to the platform.

Bridging Restaurants and Consumers:

The app offers a unique space for restaurants to showcase their offerings and interact directly with their audience. By fostering authentic user-generated content, the app helps businesses gain visibility and connect with potential customers organically.

Creating Value for Both Users and Businesses:

By integrating reward mechanisms, such as discounts and promotions for active contributors, the app not only incentivizes engagement but also creates tangible value for its users. Restaurants benefit from increased visibility and targeted marketing, while users enjoy exclusive perks for their participation.

Creating a sustainable and powerful advertising platform for Businesses:

According to a study, it was revealed that online promotions significantly impact consumers' sharing of restaurant posts [6] "As such, this project would create a platform that enables restaurants to harness the power of user-generated

content and social engagement. By integrating online promotions with authentic consumer experiences, this project aims to drive greater visibility for restaurants and foster trust among users. Engagement in sharing restaurant posts online is a strong predictor of online food purchasing intentions [7] which highlights the potential for increased customer engagement and sales. This project will focus on developing a seamless experience for both food enthusiasts and restaurants, enabling the effective sharing of food-related content while providing businesses with targeted advertising tools to enhance their reach and build lasting customer loyalty."

Conclusion:

In essence, this app combines the utility of food review platforms with the engagement of social media, creating an ecosystem that benefits all stakeholders. It seeks to revolutionize the way food is discovered, shared, and celebrated, making it the ultimate digital destination for food lovers and businesses alike.

1.3 Project Aim

The primary aim of this project is to develop an innovative food review and social media application that bridges the gap between consumers and restaurants while fostering a dedicated community of food enthusiasts. The platform combines the best of both worlds—offering users a space to share their dining experiences, post food photos, and connect with friends and like-minded individuals. By integrating social connectivity with authentic reviews, the app enhances the dining experience, allowing users to discover and engage with food content in a more interactive and rewarding way.

Recognizing that small business owners often have limited knowledge regarding marketing and may lack the time or patience to learn effective strategies (Hogarth-Scott et al., 1996) [8], the platform simplifies marketing for restaurants by providing them with a dedicated, targeted space for advertisement and audience engagement. This enables restaurant owners to reach their ideal

audience with minimal effort, while fostering a mutually beneficial environment for both consumers and both small and larger businesses.

1.4 Objectives

Create a User-Centric Food Review Platform:

Develop features that allow users to share detailed reviews, ratings, photos, and videos of their dining experiences. Ensure the platform emphasizes authenticity and transparency in user-generated content.

Integrate Social Media Functionalities:

Enable users to connect, follow, and interact with other food enthusiasts, fostering a vibrant online community. Incorporate features like feeds, comments, likes, and private messaging to enhance social interaction

Provide Incentives and Rewards:

Implement a reward system to encourage users to actively post reviews, share content, and engage with the platform. Partner with restaurants to offer discounts or perks for highly active contributors.

Empower Restaurants with Targeted Marketing Tools:

Design dedicated spaces for restaurants to showcase their promotions, and events.

Design a quest system that encourages users to engage with restaurants.

Facilitate Seamless Discoverability:

Implement a filter system that helps users find restaurants based on preferences, location, and trending reviews. Integrate advanced filtering options for cuisine types, pricing, and dietary requirements.

Ensure a User-Friendly Application: Prioritize a seamless and intuitive user interface to enhance usability for both consumers and restaurants.

Scalability:

Utilize scalable technology to accommodate growth and ensure consistent performance under increased user demand.

Promote Community Engagement and Trust:

Introduce moderation and review verification processes to maintain the quality and reliability of user-generated content. Foster a welcoming and inclusive environment where users feel encouraged to share and connect.

Chapter 2: Technical Documentation

2.1 Project Scope

Project scope is the intended scope of the project for illustration purposes, based on the initial project plan. Certain scopes were not met.

2.1.1 User features

User Profiles:

Allow users to create and manage personal profiles, including uploading profile pictures, bios, and tracking their reviews and activities.

Food Reviews and Ratings:

Users can write reviews and rate restaurants.

Social Interaction:

Implement features such as adding buddies, liking and commenting on posts to foster social connections within the food community.

Feed and Discoverability:

Design an interactive feed where users can discover friends posts, check ins and reviews.

Explore:

Design an explore section where users can view non-buddy content from other users who did not private their accounts.

Reward System:

Introduce an incentive-based system where active users can earn rewards like discounts or special offers from partnered restaurants for their contributions (e.g., posting reviews, sharing content).

Tier System: Introduce a system where users can increase their tier/rank to get 'titles' which are displayed on their profile.

Level system: Introduce a system where users can get exp points by completing quests which would increase their level. Their level is displayed on their profile and viewable by other users.

2.1.2 Restaurant features

Advertisement Space:

Provide restaurants with dedicated spaces within the app to promote new menu items, special deals, and events to the target audience.

Restaurant Profiles:

Restaurants can create and manage profiles that showcase their menus, photos, special promotions, and events.

Analytics and Insights:

Implement a dashboard for restaurants to view analytics on user engagement, reviews, and customer behavior to inform their marketing strategies.

Engagement with Users: Allow restaurants to respond to reviews, engage with users via comments, and offer personalized discounts or deals to users based on their interactions.

2.1.3 Technical Requirements

Cross-Platform Mobile App:

Develop the application for both iOS and Android platforms using the Expo framework.

Backend Infrastructure:

Set up a robust backend using Flask to handle user data, restaurant information, review data, promotions, messages, posts etc.

Database Management:

Implement a relational database using MySQL to store user profiles, reviews, ratings, restaurant data, and interactions.

Scalability and Performance:

Ensure that the application is scalable to handle increasing numbers of users and restaurants while maintaining optimal performance. (Such as being able to upgrade from free plans to paid plans).

2.1.4 Cloud Storage for Image upload/retrieval

Image/Video Storage and Retrieval Supabase Integration:

Use Supabase for the secure storage and retrieval of images and videos uploaded by users and restaurants. Supabase provides a scalable and reliable solution for handling multimedia content, ensuring quick access and smooth performance within the app. The integration will include functionality for users to upload food

photos and videos with their reviews, as well as restaurants showcasing their menu items and promotions.

2.1.5 Search Functionality and Autofill

Elasticsearch Integration:

Implement Elasticsearch for handling advanced search queries and enhancing the user experience with fast, accurate search results. This will enable users to search for restaurants, food items, and other users, with support for real-time autofill suggestions. Additionally, Elasticsearch will improve the efficiency of finding friends, reviews, or food-related content, making it easier for users to discover relevant posts, connect with others, and engage with the community. The platform will provide users with an intuitive and responsive search interface that helps them navigate the app effortlessly.

2.2 Application Development

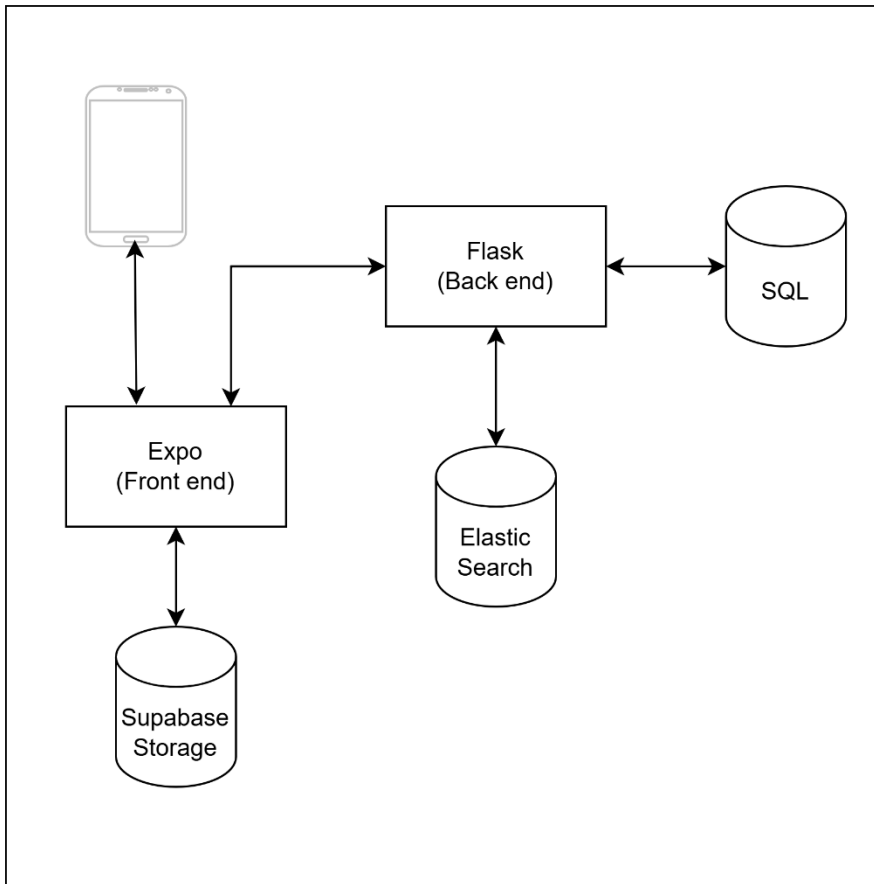


Figure 1 - Overview of application Structure

2.2.1 Overview of application structure

Components: Expo, Supabase, Elasticsearch, Flask, My SQL, IOS/Android Phone

Description:

The mobile application, built with Expo, is directly integrated with the backend services. Depending on user interactions, GET or POST requests are sent to a Flask server, which handles data processing and communicates with the underlying data stores—Elasticsearch and SQL databases. For GET requests, the server retrieves the relevant data, processes it, and sends it back to Expo, where it is rendered and displayed within the mobile application.

Additionally, Expo interacts directly with Supabase Storage for media handling, allowing images and videos to be accessed via their URIs. This eliminates the need for media conversion to byte format, enabling efficient and seamless media rendering within the app.

2.2.2 Front End Development

Bitespace is built using Expo, a versatile framework that enables efficient cross-platform mobile app development for both Android and iOS. Expo simplifies the development process by providing an extensive collection of pre-built libraries, APIs, and tools, significantly minimizing the time and effort needed to incorporate standard mobile app features. These include capabilities such as push notifications, camera access, and location services, all integrated directly into the development environment. With its comprehensive ecosystem, Expo supports rapid prototyping, continuous development, and straightforward deployment, eliminating the need for complicated configurations.

The application is developed with TypeScript, a statically typed extension of JavaScript that improves code quality and boosts developer efficiency. By leveraging TypeScript's static typing, errors are detected during compilation rather than at runtime, reducing bugs and enhancing the stability of the application. TypeScript also offers superior tooling, such as advanced autocompletion, type inference, and refactoring support, which accelerates development and helps maintain a well-structured, scalable codebase.

By utilizing Expo and TypeScript, Bitespace takes advantage of a contemporary development workflow that improves maintainability and scalability, guarantees code quality through type safety, and speeds up cross-platform development. Expo's user-friendly interface and integrated features enable smooth integration with native device functionalities, while TypeScript's type system ensures the app remains reliable and adaptable as new features are incorporated.

2.2.2.1 Pros and Cons of Expo

Advantages

Cross-Platform Development:

Expo enables developers to create a single codebase that functions seamlessly on both Android and iOS platforms, reducing the time and effort required for development. Some minor issues to consider would be small differences between iPhone and Android.

E.g. SafeAreaView (Ensures content is rendered beneath the status bar) works slightly differently on iOS as compared to android.

Accelerated Development with Pre-built Components:

Expo offers a comprehensive collection of pre-built components and libraries, facilitating faster development without the need to create custom solutions from scratch.

Simplified Setup and Management:

Expo streamlines the setup process and handles key aspects of app development, such as bundling, building, and deployment, allowing developers to focus on writing code.

OTA (Over-the-Air) Updates:

Expo supports Over-the-Air updates, enabling developers to deliver updates directly to users without requiring them to download a new version from the app store.

Disadvantages

Learning Curve:

Although Expo streamlines mobile development, developers must still learn its unique workflows and libraries, which consumed a lot of time in this project.

Constant updates:

Throughout the course of this project, Expo has undergone several updates, presenting a range of challenges. These frequent updates often introduce breaking changes or discontinue certain features, forcing the team to adjust the codebase to ensure ongoing compatibility. This leads to additional development time spent on tasks like updating dependencies, resolving bugs, and modifying the code to work with updated APIs. In some instances, major updates necessitate thorough testing and validation of existing functionality to ensure everything continues to function correctly, which can cause delays in the project timeline. Moreover, staying current with Expo's continually evolving ecosystem may restrict the use of older libraries or specific native modules, which can limit the project's flexibility. The mobile Expo Go also has to be compatible with the Expo version used in the creation of the app. As such, I was forced to constantly update both ends.

2.2.2.2 Pros and Cons of Typescript

Advantages

TypeScript Static Typing:

TypeScript's static typing detects errors at compile time rather than at runtime, reducing bugs and improving the overall quality of the code.

Enhanced Developer Experience:

With features such as autocompletion, advanced refactoring tools, and powerful IntelliSense, TypeScript boosts developer productivity and simplifies code maintenance.

Improved Readability and Maintainability:

TypeScript's clear type definitions enhance code readability, making it easier for teams to collaborate and understand the code's structure and behavior.

Disadvantages

Learning Curve:

TypeScript can be challenging for developers familiar with JavaScript's dynamic typing, as it requires learning its static typing system and understanding type definitions.

Reduced Flexibility with JavaScript:

While TypeScript is built on JavaScript, its strict typing system can restrict the flexibility that developers accustomed to JavaScript's dynamic nature might expect, especially when dealing with untyped code.

2.2.3 Backend Development

The backend of the application is developed using Flask, a lightweight and adaptable Python web framework. Flask was chosen for its simplicity, scalability, and its capacity to rapidly develop RESTful APIs. Its minimalist approach offers flexibility, allowing for customization without enforcing rigid structures, making it perfect for projects that require quick iterations and easy integration with various services.

Flask's built-in development server, along with its robust routing capabilities, facilitates efficient handling of HTTP requests. It supports a wide range of HTTP methods, including GET, POST, PUT, and DELETE, enabling seamless interaction between the frontend and backend for tasks such as data retrieval, creation, updates, and deletions.

2.2.3.1 Setup

1. The backend development begins with setting up Flask and integrating essential extensions like SQLAlchemy for database management.

2. Environment Initialization: A virtual environment was created to manage dependencies, ensuring isolation and preventing conflicts between libraries.
3. Flask Application Configuration: The Flask instance was initialized and configured with necessary settings such as Database URI for SQLAlchemy. Secret keys for session management. Additional configurations for debugging and testing modes.
4. SQLAlchemy Integration: SQLAlchemy was initialized to handle interactions between the application and the database. It serves as the ORM, providing an abstraction layer over raw SQL queries.
5. Extensions Setup: Additional Flask extensions such as Flask-CORS (for enabling cross-origin requests) were also integrated.
6. Initialization of Elasticsearch: Elasticsearch was integrated into the backend to provide efficient full-text search and advanced data retrieval capabilities. Elasticsearch was installed and configured on the server, with appropriate settings for indices and search performance. The elasticsearch-py library was used to connect the Flask application to the Elasticsearch server.

The Elasticsearch client was initialized in the backend with connection parameters like host, port, and authentication details. Connection was tested to ensure the server is reachable and ready for indexing and querying data.
7. Listeners for Database Changes: Event listeners were added to track changes in the database (e.g., insertions, updates, deletions). When changes occur, the corresponding data is updated in Elasticsearch to maintain synchronization with the primary SQL database.
8. Manual Upload for Testing: A script (ElasticsearchManualUpload.py) was developed to upload data manually for testing purposes. This ensured that sample data could be indexed in Elasticsearch without requiring a fully operational application.
9. Connection with database: The SQLALCHEMY_DATABASE_URI configuration parameter was set in the Flask application to specify the connection string for the database.

2.2.3.2 Models

Model Integration: Models were defined using SQLAlchemy, allowing seamless interaction with the database.

2.2.3.2.1 Models Overview

Restaurants: Represents restaurant entities, including their basic information and attributes.

OutletDetails: Captures details about individual outlets of a restaurant, such as location and promotions.

Clients: Stores client information, including user credentials, profile information, and preferences.

Reviews: Tracks reviews submitted by clients for outlets.

Checkin: Records client check-ins at specific restaurant outlets, including timestamps and locations.

Posts: Represents user-generated content, such as social media-style posts.

Likes: Tracks user likes on posts, reviews, or other content.

Comments: Records comments on posts or reviews, facilitating discussions.

BuddyList: Manages client-to-client connections for social networking within the app.

Quests: Tracks user participation in app-specific challenges or gamified activities.

CompletedQuests: Tracks if quests are completed by users.

Promotions: Represents promotional campaigns run by outlets.

Messages: Handles user-to-user messaging within the app.

2.2.3.2 Relationships and Interactions

The models are designed with relationships to enforce associations between different entities.

For example: One-to-Many: A restaurant (Restaurants) can have multiple outlets (OutletDetails).

Many-to-Many: Clients can follow multiple restaurants, and restaurants can be followed by multiple clients (BuddyList).

The models not only define the structure of the tables but also include methods and queries to handle business logic, such as retrieving a client's buddies, listeners to update Elasticsearch whenever a table is updated or calculating the popularity of a restaurant based on reviews.

2.2.3.3 Routes

Routes act as access points for managing HTTP requests and facilitating communication between the expo frontend, SQL database and Elasticsearch. Each route is associated with a specific endpoint and functionality, determining how the application handles different HTTP methods, thereby ensuring smooth interaction with the backend. Only GET and POST methods were required in this project.

2.2.3.3.1 Test routes

After creating and importing each model into app.py, GET routes were implemented to verify successful data retrieval from the database. These test routes were created for manual testing on the browser and ensured that the backend was properly interacting with the database, facilitating early detection of issues related to data access.

For example:

```
# for testing
```

```
@app.route('/api/outletdetails')
def get_outletdetails():
    outlets = OutletDetails.query.all()
    return jsonify([
        'outlet_id': outlets.outlet_id,
        'restaurant_id': outlets.restaurant_id,
        'address': outlets.address,
        'location': outlets.location,
        'likes': outlets.likes,
        'pic': outlets.pic,
        'featured': outlets.featured,
        'promotion': outlets.promotion
    ]for outlets in outlets])
```

This route queried the OutletDetails model to retrieve data from the outlet_details table in the SQL database. The results were displayed in the browser in JSON format, confirming that the data was being fetched correctly. This process helped ensure that any issues related to the model's ability to interact with the database could be promptly identified and addressed, thus ruling out data access as an issue during later stages of development. By testing early, I could ensure smoother integration and avoid later complications related to data retrieval.

2.2.3.3.2 GET routes

GET routes were extensively used in this project to render various components, particularly when no data input was required from the frontend to fetch the necessary data. These routes were essential for efficiently retrieving and displaying information from the backend.

For example, the following route was used to fetch data for the "Explore" section:

```
#Route for explore
@app.route('/explore', methods=['GET'])
def explore():
```

```

# Step 1: Find all public clients (private = 0)
public_clients = Clients.query.filter_by(private=0).all()

# Get client IDs of public users
public_client_ids = [client.client_id for client in public_clients]

if not public_client_ids:
    return jsonify({"error": "No public users available"}), 404

# Step 2: Retrieve posts from these public clients within the last 3 days
three_days_ago = datetime.now(timezone.utc) - timedelta(days=20)
posts = db.session.query(
    Posts.post_id.label('post_id'),
    Posts.client_id,
    Posts.post_time,
    Posts.post_title,
    Posts.image_url,
    Clients.client_name,
    Clients.profilepic,
).join(Clients, Clients.client_id == Posts.client_id) \
    .filter(Posts.client_id.in_(public_client_ids), Posts.post_time >=
three_days_ago) \
    .all()

posts_data = []

# Step 3: Collect data from Likes and Comments tables
for post in posts:
    # Retrieve the number of comments and likes for this post
    comment_count = Comments.query.filter_by(post_id=post.post_id).count()
    like_count = Likes.query.filter_by(post_id=post.post_id).count()

    # Prepare the data for each post with client details
    post_data = {
        'client_id': post.client_id,
        'client_name': post.client_name,
        'profile_picture': post.profilepic,
        'post_id': post.post_id,
        'post_time': post.post_time.isoformat(),
        'post_title': post.post_title,
        'image_url': post.image_url,
        'comment_count': comment_count,
        'like_count': like_count,
    }

```

```
posts_data.append(post_data)

# Return the compiled posts data as JSON
return jsonify(posts_data), 200
```

This route was designed to fetch and compile the necessary data for the "Explore" section of the application. Specifically, it retrieves public clients (those with `private = 0`), then gathers posts from these users within the last 3 days. It also collects additional data from the Likes and Comments tables to display the number of interactions with each post.

The compiled data is sent to the frontend as a JSON response, allowing the application to dynamically render post cards in the "Explore" section of the newsfeed tab. This approach ensures that the explore section is populated with relevant, up-to-date content while minimizing the need for excessive client-side data handling.

2.2.3.3.3 POST routes

POST routes were utilized to handle data sent from the frontend, enabling the extraction of specific data from the database. These routes facilitate the creation of new entries or the updating of existing ones based on the data provided by the client-side application.

For example:

```
#Route for user posts
@app.route('/userpost', methods=['POST'])
def create_post():
```

```

try:
    # Get JSON data from the request
    data = request.get_json()

    # Validate incoming data
    if not data.get('username') or not data.get('post_title') or not
data.get('image_url'):
        return jsonify({'error': 'Missing required fields'}), 400

    # Retrieve the client_id from the clients table using the username
    client = Clients.query.filter_by(username=data['username']).first()

    # Check if the client exists
    if not client:
        return jsonify({'error': 'Client not found'}), 404

    # Create a new post using the retrieved client_id
    new_post = Posts(
        client_id=client.id, # Use client_id instead of username
        post_title=data['post_title'],
        image_url=data['image_url'],
        post_time=datetime.now()
    )

    # Add the post to the database
    db.session.add(new_post)
    db.session.commit()

    # Return success message
    return jsonify({'message': 'Post created successfully!', 'post_id':
new_post.id}), 201

except Exception as e:
    # Handle any errors
    return jsonify({'error': str(e)}), 500

```

This route processes the creation of a new post by receiving the client's username, post title, and image URL from the frontend. It first validates the incoming data to ensure all required fields are present. Then, it retrieves the client_id from the database using the provided username.

If the client exists, a new post is created, associating it with the correct client_id and storing the post details such as the title, image URL, and timestamp. Once the

new post is added to the database and committed, the route returns a success message along with the post_id. If any errors are encountered during the process, an appropriate error message is returned to the frontend.

This approach enables the dynamic creation of posts on the user's profile, which will later be displayed as post cards within the application.

2.2.3.4 Elasticsearch

Elasticsearch API was used to leverage its powerful search capabilities, particularly for autocompletion. It was utilized in the search bar functionality to enhance the user experience by providing relevant client data as users type. This implementation allows for faster and more efficient searches, improving the responsiveness of the application when fetching client information for rendering on client cards.

These are the elastic search settings for the client index:

PUT /clients

```
{
  "mappings": {
    "properties": {
      "client_id": {
        "type": "integer"
      },
      "client_name": {
        "type": "text",
        "analyzer": "edge_ngram_analyzer"
      },
      "profilepic": {
        "type": "text"
      }
    }
  }
}
```

```
    }  
  }  
},  
"settings": {  
  "analysis": {  
    "tokenizer": {  
      "edge_ngram_tokenizer": {  
        "type": "edge_ngram",  
        "min_gram": 1,  
        "max_gram": 25,  
        "token_chars": ["letter", "digit"]  
      }  
    },  
    "analyzer": {  
      "edge_ngram_analyzer": {  
        "type": "custom",  
        "tokenizer": "edge_ngram_tokenizer"  
      }  
    }  
  }  
}
```


This setup enables an optimized search experience for users, allowing for quick retrieval of client data based on partial or incomplete input, especially in the context of autocomplete in the search bar.

2.2.4 SQL

MySQL served as the central data storage solution for the application, providing a robust relational database management system (RDBMS) to handle the backend data efficiently. The application's data architecture relies on MySQL for structured storage, retrieval, and management of information related to users, restaurants, social interactions, and other key functionalities.

MySQL's ability to handle large volumes of transactional data, support complex queries, and provide ACID (Atomicity, Consistency, Isolation, Durability) compliance ensures that the application's data is reliably managed and easily accessed. The database is designed to store information across multiple interrelated tables, each representing a specific entity or aspect of the application. These tables are structured to facilitate seamless interaction between the different components of the application, ensuring data consistency and integrity.

2.2.4.1 Communication Diagram

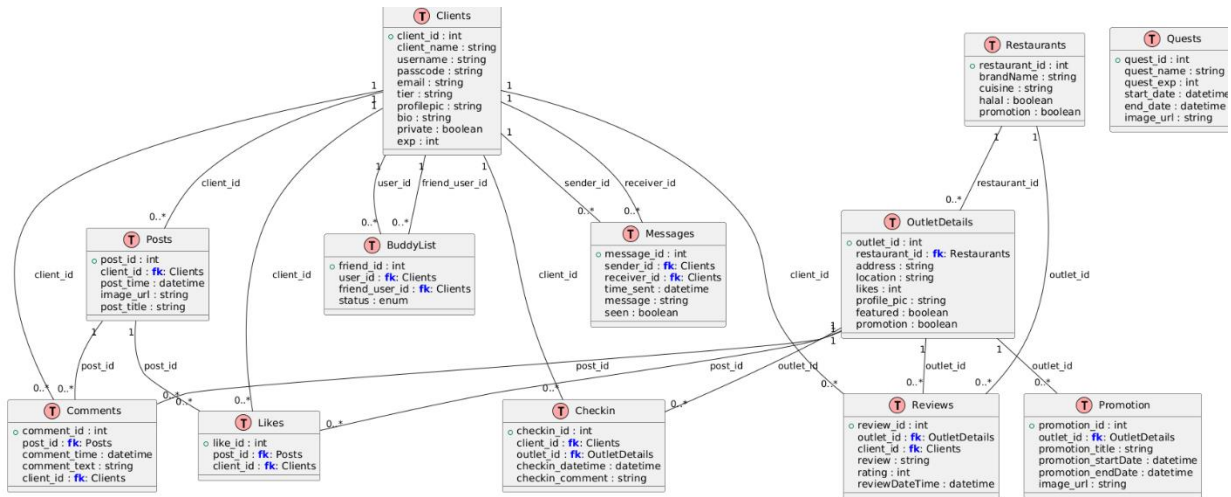


Figure 2 – Communication diagram

2.2.5 Supabase

In our implementation, Supabase directly interfaces with the frontend for seamless integration. Images associated with posts are uploaded to a Supabase storage bucket, from which a URL is generated upon successful upload. This URL is then included alongside the post details and sent to the backend for storage.

This approach eliminates the need to convert images into byte data for storage in the database, simplifying both the storage and retrieval processes. Furthermore, when rendering profile and post images, the application directly utilizes the URLs stored in the backend. This strategy avoids the computational overhead of converting image data back from bytes, resulting in improved performance and a more streamlined workflow.

2.2.6 Use Case Scenario & Use Case Diagrams (& Process Flow diagrams where necessary)

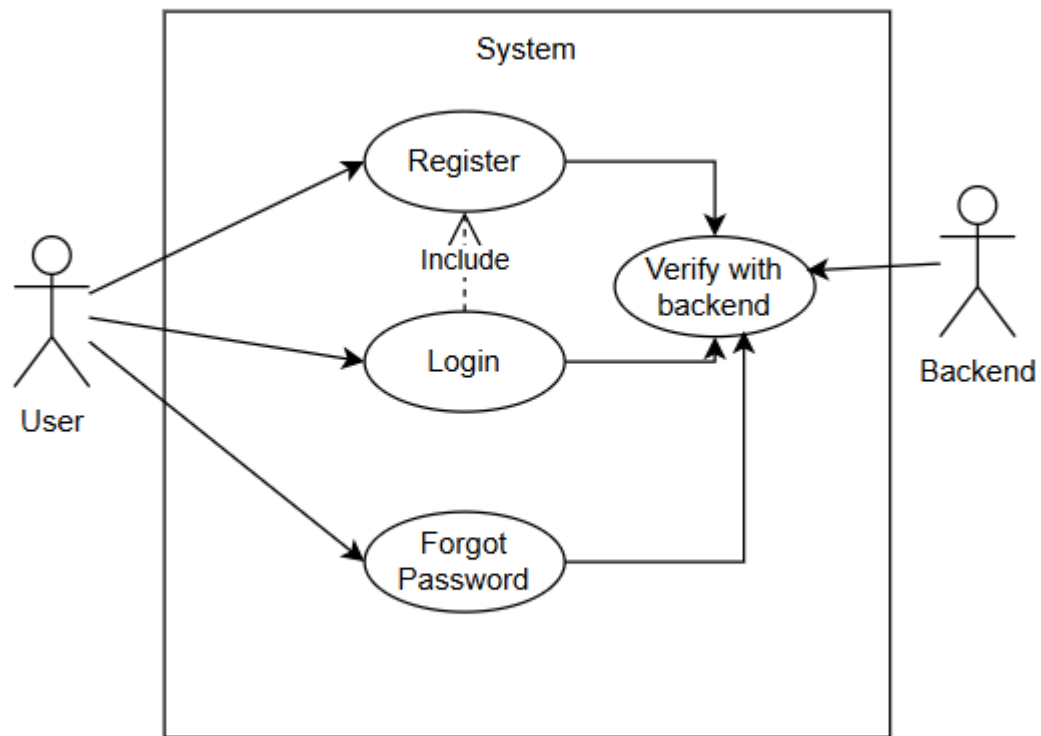
A Use Case Scenario outlines how a user interacts with the system to accomplish a specific task, detailing each step in the process. It helps to clarify user interactions and confirm that the system's functional requirements are met.

A Use Case Diagram visually represents the connections between users and system functions (use cases). It gives a broad overview of user actions, such as logging in, posting content, or checking a profile.

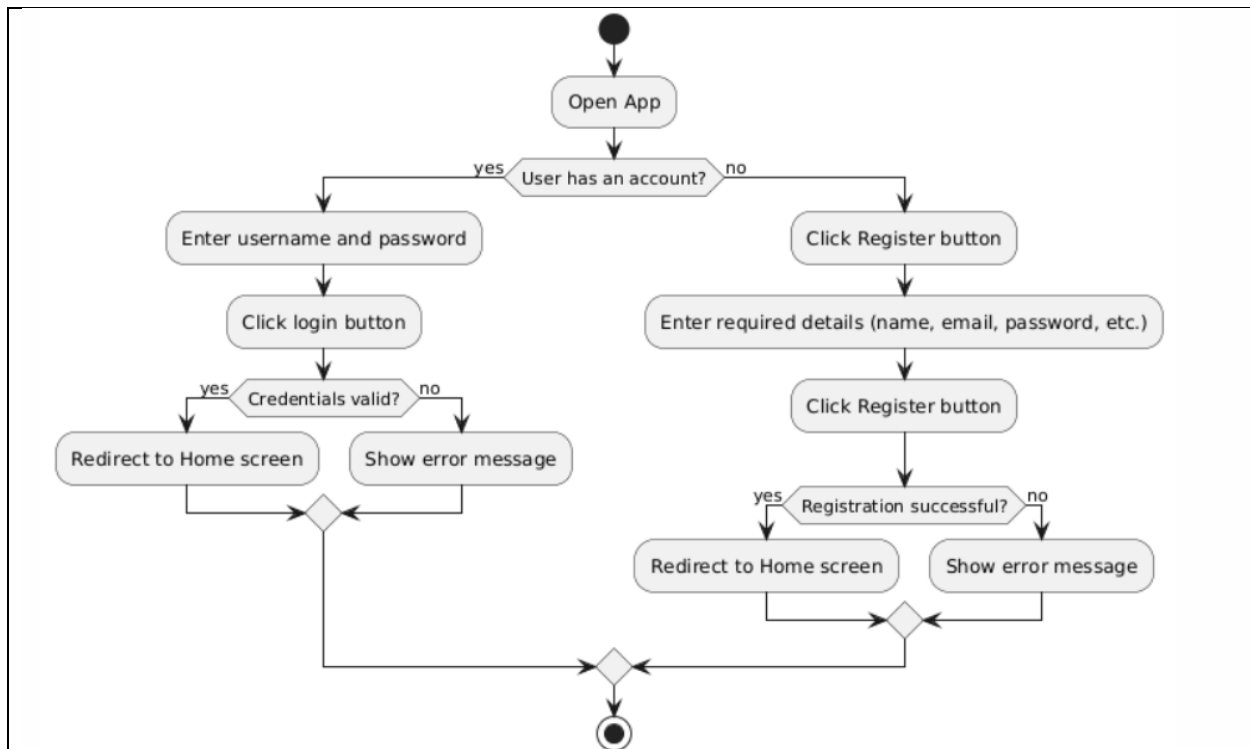
By combining the Use Case Scenario and Use Case Diagram, developers gain both a detailed understanding of user actions and a high-level view of system functionality, ensuring all requirements are addressed and potential issues are identified early.

2.2.6.1 UC-1 Login/Register

Use Case Diagram:



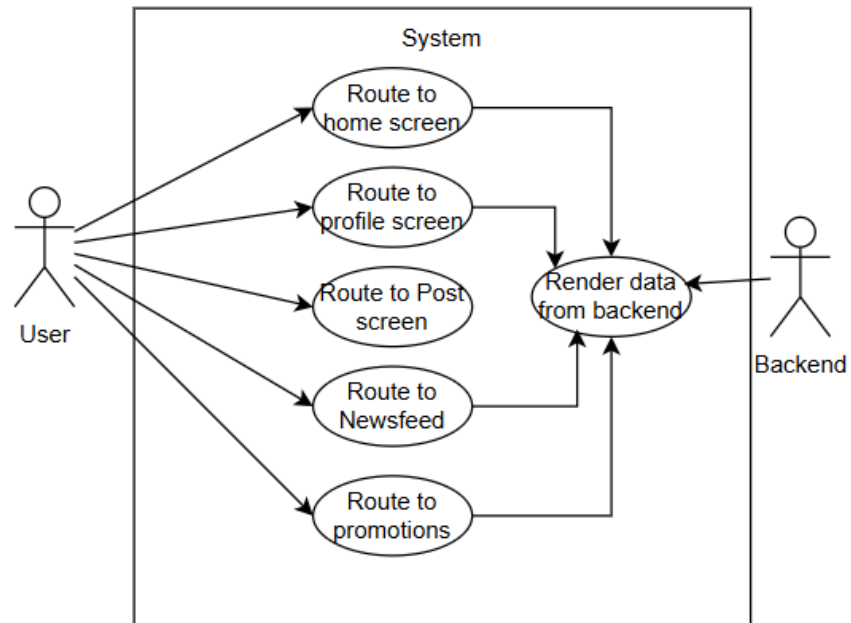
Process Flow Diagram:



Description:	To create/register for an account
Pre-Condition:	The user has to have downloaded the application The user is at the landing page
Post-Condition:	Registration: The account is successfully created, user is routed to login in page, account details are stored in database. Log in: The user is successfully verified, user is routed to logged in screen
Flow of events:	User is presented with login/register buttons If register button is clicked: Registration page is displayed. User keys in details. If username is taken, 'username is taken' message. If successful registration, route to login page. If Login button is clicked: Login page is displayed. User keys in username and password. If authenticated, routes to logged in screen. Else, Invalid credentials (username/password) message is displayed.

2.2.6.2 UC-2 Logged in screen

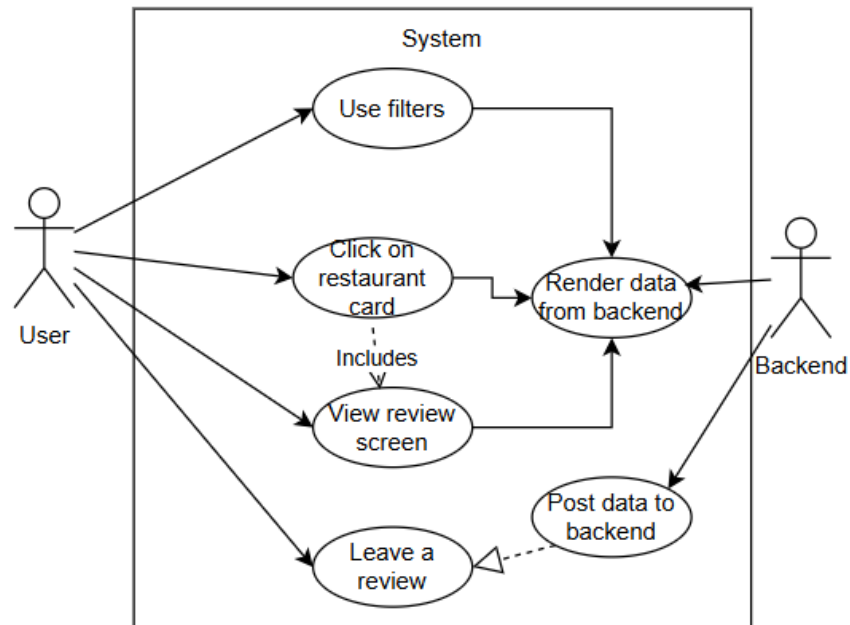
Use case diagram:



Description:	Routing through the different tabs
Pre-conditions:	User must be logged in
Post-conditions:	User routes to tab after clicking the tab buttons. Tab is populated with data from the backend.
Flow of Events:	User clicks on a tab. The screen changes.

2.2.6.3 UC-3 Home Tab

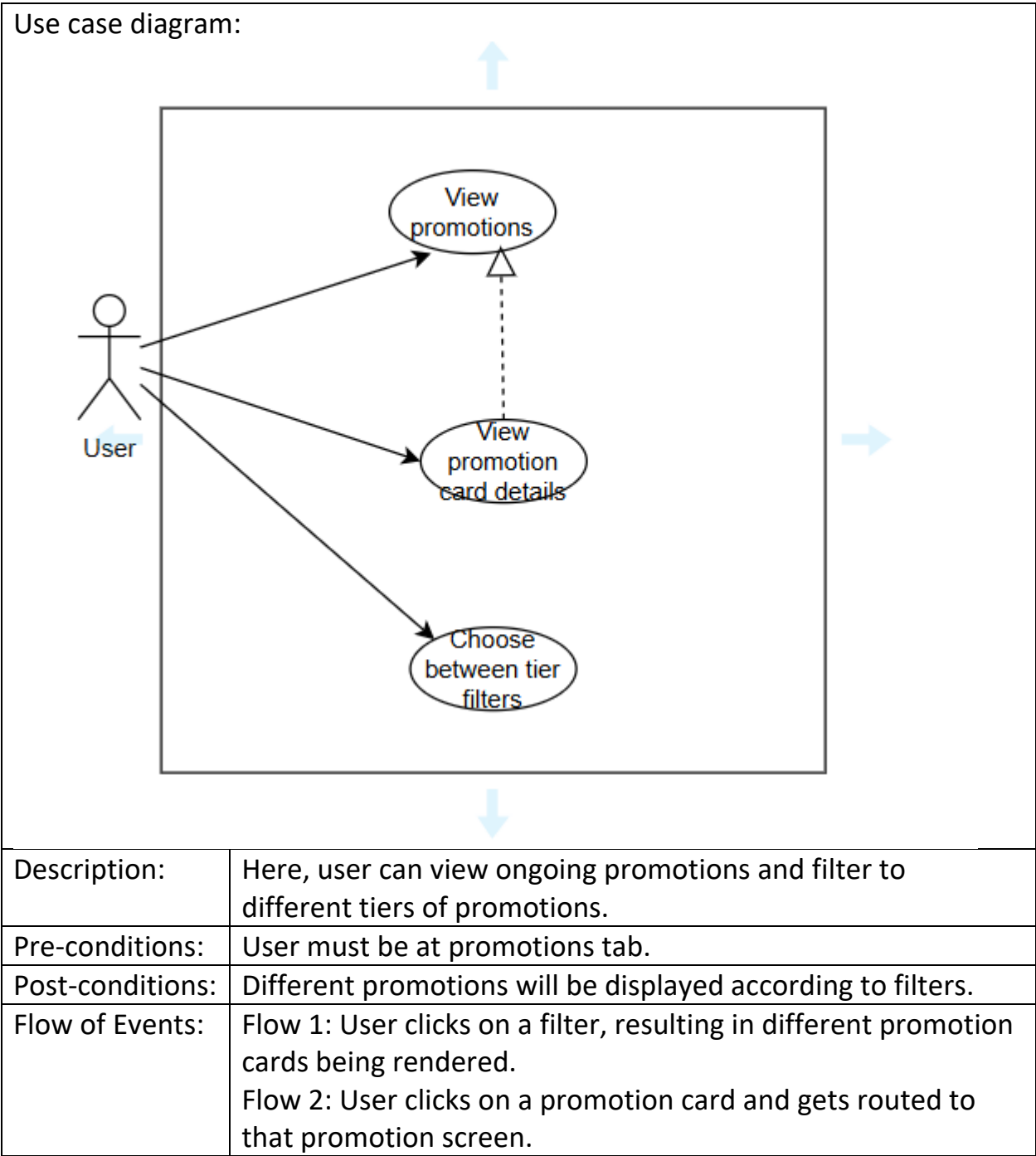
Use case diagram:



Description:	Home tab displays restaurant cards with different filters and restaurant cards. All of which can be clicked.
Pre-conditions:	User must be at the home tab.
Post-conditions:	On filter click: Restaurant cards would be filtered and displayed. On restaurant card click: Route to review screen for that restaurant. Leave review: Review data is stored in database and rendered on review card.
Flow of Events:	If user clicks on a filter. Restaurants are filtered based on filter type. If user clicks on a restaurant. They are routed to review screen where they can leave a review. User leaves a review and presses post.

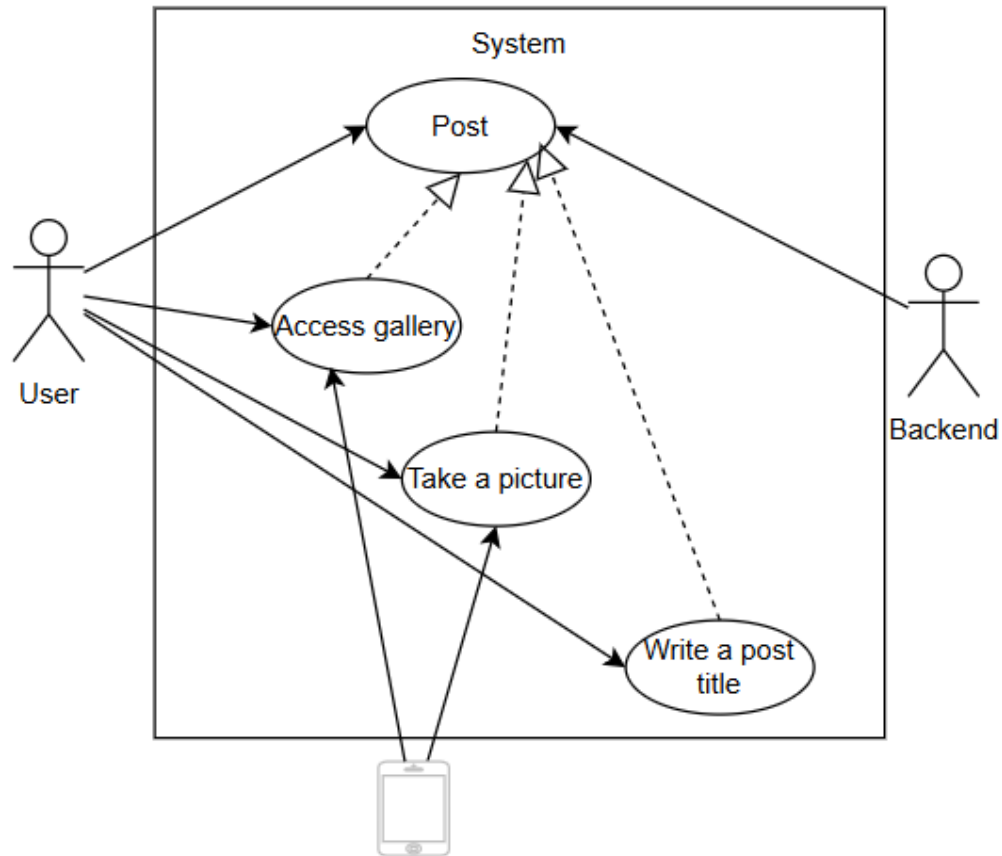
	Review data is stored in the database. Review card would show upon refreshing the screen.
--	--

2.2.6.4 UC-4 Promotions Tab



2.2.6.5 UC-5 Post Tab

Use case diagram:



Description:	Post tab for user to create posts.
Pre-conditions:	User must be at the post tab.
Post-conditions:	Image is uploaded to cloud storage, url is retrieved, url + post details are stored in database.
Flow of Events:	User writes a post title. User chooses between choose a picture from gallery/take a picture. User clicks on post.

	Image is sent to cloud storage, url for the image is retrieved and sent together with post title and username to backend, which transforms the data, includes other necessary data and updates the database.
--	--

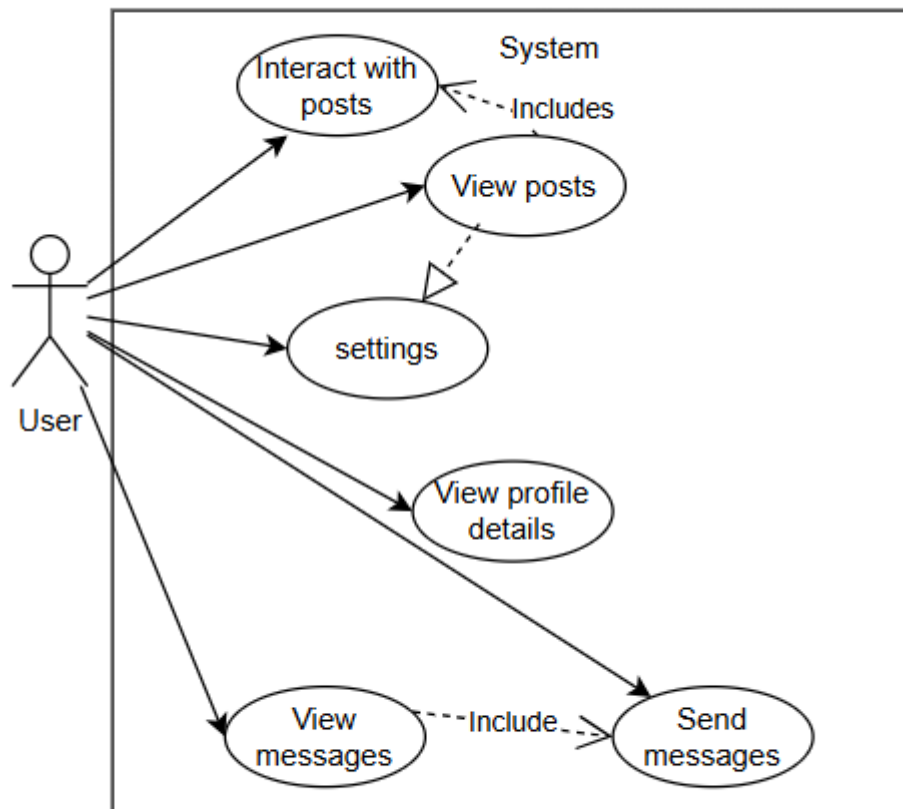
2.2.6.6 UC-6 Newsfeed Tab

Use case diagram:	
The diagram shows a system boundary containing several use cases. A 'User' actor is connected to 'Interact with posts', 'View posts', 'choose friends/explore tab', 'Perform search', and 'Navigate to searched profile'. A 'Backend' actor is connected to 'Retrieves data from backend'. Inside the system, 'View posts' includes 'Interact with posts' (dashed arrow). 'choose friends/explore tab' includes 'View posts' (dashed arrow). 'Perform search' includes 'Navigate to searched profile' (dashed arrow). 'Retrieves data from backend' includes 'View posts' (solid arrow). 'Navigate to searched profile' includes 'Retrieves data from backend' (solid arrow).	
Description:	Newsfeed tab where users can toggle between friends posts and explore. User can click on search bar to search for other users, and get routed to their profile. Here, user can view

	their profile details and see their posts or add as a buddy. User can route to other user's profile from the search bar. User can interact with the posts.
Pre-conditions:	User must be on the newsfeed tab.
Post-conditions:	User will be routed to profile screen of other users. If like or comment on a post, data would be sent to the backend to be committed to the database. Screen will change upon click button click 'friends' or 'explore'. If pressed 'add buddy', buddy request is sent.
Flow of Events:	Flow 1: User clicks on friend's tab and only friend's posts are displayed. Flow 2: User clicks on explore tab and all not private posts are displayed. Flow 3: User clicks on search bar, types a search. Profiles are displayed. User can click on a profile and gets routed to the user's profile page. User can add buddy or if already added, button will display 'buddied'.

2.2.6.7 UC-7 Profile Tab

Use case diagram:



Description:	Here, users can view their own posts, messages, profile details, buddylist, level. They can click on buddylist, settings, level and be routed to the respective pages.
Pre-conditions:	User must be at profile tab.
Post-conditions:	Comments/likes/messages/settings are sent to backend and committed to database.
Flow of Events:	Flow 1: User interacts with post, interaction is sent to backend. Flow 2: User presses on settings/messages/level/buddylist and is routed to the page.

2.3 Figma Designs & Test cases

This section presents the Figma designs alongside the corresponding test cases. The testing approach employed was End-to-End (E2E) manual testing, facilitated by Expo's real-time update capabilities. This enabled efficient simultaneous testing of both backend routes and frontend screen rendering. The test results were displayed directly on the screens, and these results, along with the Figma designs, will be showcased in this section.

2.3.1 Landing Screen

Test case: Landing Screen should be rendered upon successful bundling of the app.

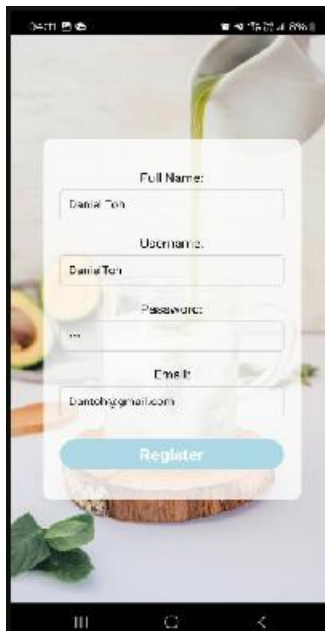
Results: Success



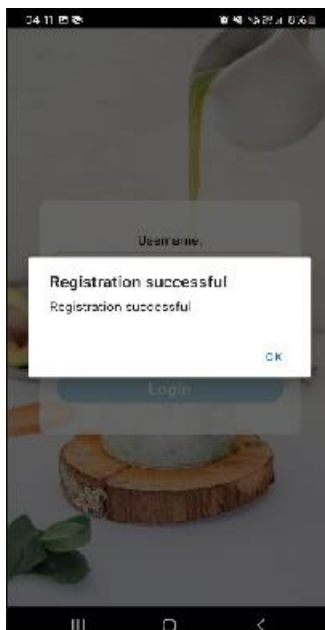
2.3.2 Registration

This section would include all tests and figma designs under registration.

Test case: Registration Screen should be rendered upon clicking 'Register'.
Results: Success



Test case:
If username is not taken, registration should be successful.
Result: Success



Test case: This should result in the database as well as Elasticsearch being updated with the entry.

Result: Success!

Elasticsearch:

```
127.0.0.1:9200/clients/_search?size=1000&pretty

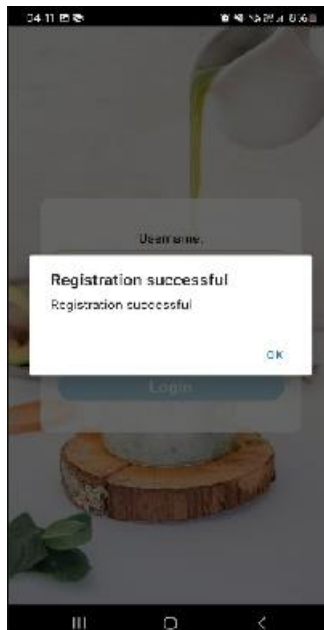
{
  "took": 724,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 2,
      "relation": "eq"
    },
    "max_score": 1.0,
    "hits": [
      {
        "_index": "clients",
        "_id": "1",
        "_score": 1.0,
        "_source": {
          "client_id": 1,
          "client_name": "Daniel lim",
          "profilepic": "https://adybybuoqeunggekybz1.supabase.co/storage/v1/object/public/BiteSpace/c0.jpg?t=2024-11-06T15%3A52%3A05.318Z"
        }
      },
      {
        "_index": "clients",
        "_id": "2",
        "_score": 1.0,
        "_source": {
          "client_id": 2,
          "client_name": "Daniel Toh",
          "profilepic": "https://adybybuoqeunggekybz1.supabase.co/storage/v1/object/public/BiteSpace/c0.jpg?t=2024-11-06T15%3A52%3A05.318Z"
        }
      }
    ]
  }
}
```

Sql:

	client_id	client_name	username	passcode	email	tier	profilepic	bio	private	ex
▶	1	Daniel lim	Danielim	123	Dandan@gmail.com	bronze	https://adybybuoqeunggekybz1.supabase.co/st...	HULL	0	0
	2	Daniel Toh	DanielToh	123	Dantoh@gmail.com	bronze	https://adybybuoqeunggekybz1.supabase.co/st...	HULL	0	0
•	HULL	HULL	HULL	HULL	HULL	HULL	HULL	HULL	HULL	HULL

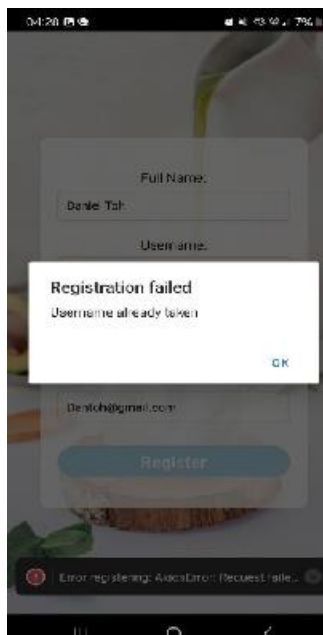
Test case: There should also be a pop up 'Registration successful!'

Result: Success



Test case: If username is taken, registration would be unsuccessful and pop up 'Username taken!'

Result: Success



Test case: Upon successful registration, user should be routed to the home screen.

Result: Success

2.3.3 Login

This section would include all test cases and figma designs under login.

Test case: Login screen should render when 'login' button is clicked.

Result: Success



Test case: User should receive invalid credentials message if username is incorrect

Result: Success



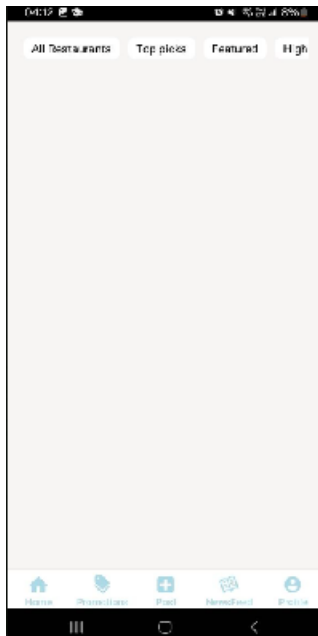
Test case: User should receive invalid credentials message if password is incorrect

Result: Fail

Description: It allows any password to log in as long as username is valid.

Test case: User should be routed to logged in page upon successful login

Result: Success



2.3.4 Tab layout

Test case: User should be routed to the different screens when tapping on the tabs.

Result: Success

2.3.5 Home Screen

This section would include all test cases and figma designs under Home Screen.

Test case: All restaurant cards should be displayed if no filter is selected

Result: Success



Test case: All restaurants should be displayed if 'All restaurants' filter is selected

Result: Success

Test case: Only halal restaurants should be displayed if 'Halal' filter is selected

Result: Success



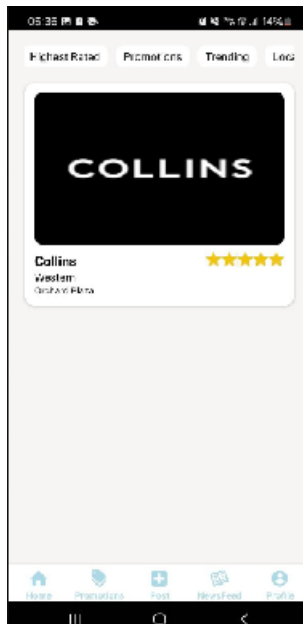
Test case: Only featured restaurants should be displayed if 'featured' filter is selected

Result: Success



Test case: Only restaurants with ongoing promotions should be displayed if 'Promotions' filter is selected

Result: Success



Test case: Only restaurants in the West should be displayed if 'West' filter is selected

Result: Success



Test case: Only restaurants in the North should be displayed if 'North' filter is selected

Result: Success



Test case: Only restaurants in the South should be displayed if 'South' filter is selected

Result: Fail

Description: No restaurant card is rendered

Test case: Only restaurants in the East should be displayed if 'East' filter is selected

Result: Success



Test case: Only restaurants in the Central should be displayed if 'Central' filter is selected

Result: Success



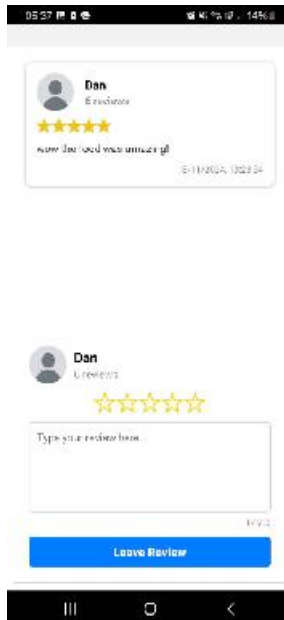
Test case: User should be routed to the Review screen for that restaurant when restaurant card is tapped.

Result: Success

2.3.6 Review Screen

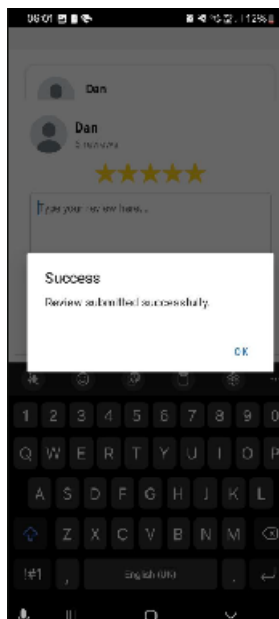
Test case: Review Screen should display reviews for the restaurant.

Result: Success



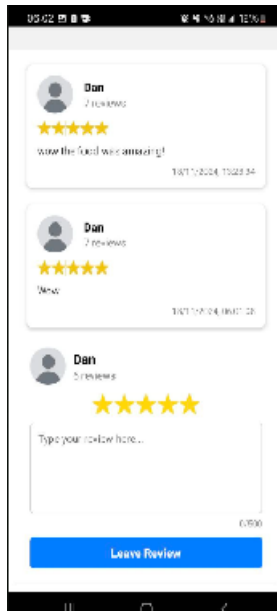
Test case: Review submitted successfully message upon successful review

Result: Success



Test case: Upon refreshing review screen, new review should appear

Result: Success



Test case: Leave review should commit the review data to database

Result: Success

26 2 1 Wow 5

2.3.7 Post Screen

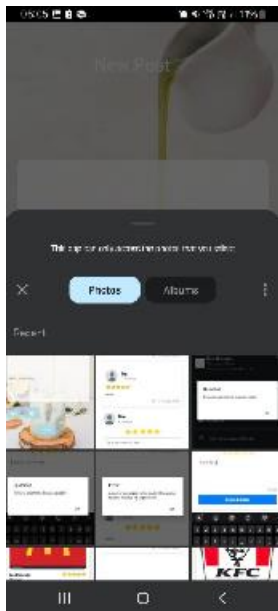
Test case: Post Screen should render upon navigation

Result: Success



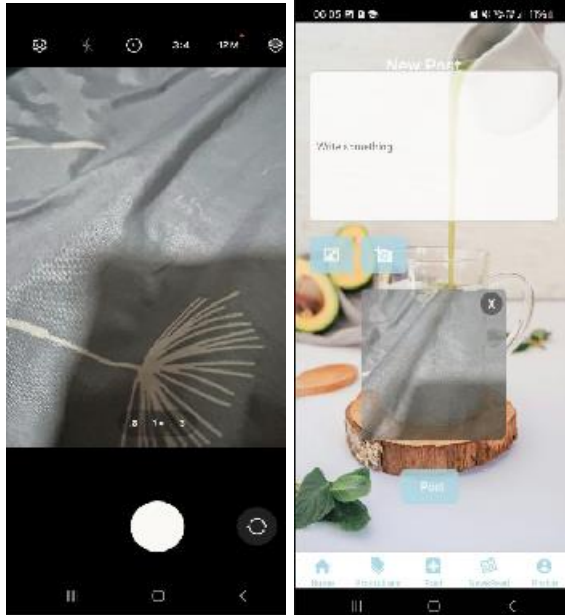
Test case: Gallery button should show phone gallery to allow user to upload image from phone

Result: Success



Test case: Take photo button should access the camera and allow user to take a photo

Result: Success



Test case: Upon clicking 'post', Image should be uploaded to supabase storage and the image storage URL and other post details should be sent do backend to be committed into database

Result: Fail

Description: Network error with supabase.

2.3.8 Newsfeed Screen

Test case: News feed screen should render upon navigation to news feed tab

Result: Success

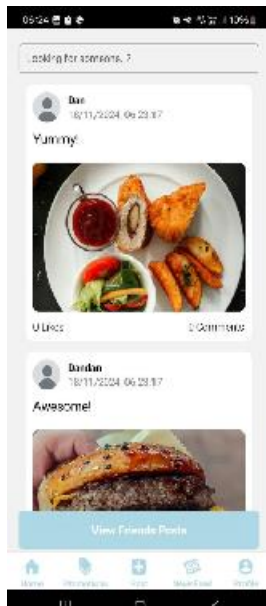
Test case: Tapping the 'Friends' Button should render Friends post cards only, and change the button to 'Explore'

Result: Success



Test case: Tapping the 'Explore' button should render all posts that are not private, and change the button to 'Friends'

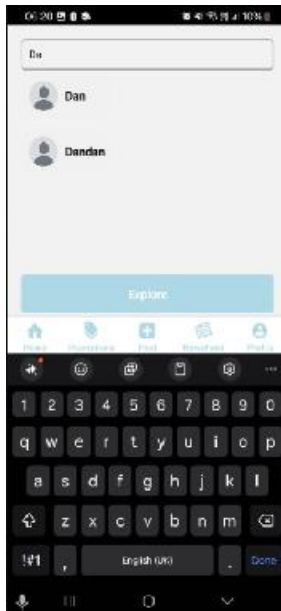
Result: Success



2.3.8.1 Search for friends

Test case: Users should be displayed when user types on search bar, with auto fill function.

Result: Success



2.3.8.2 Routing from Newsfeed to client profiles

Test case: User should be routed to other profiles when clicking on the client card on the search bar

Result: Success

2.3.9 Other Profiles

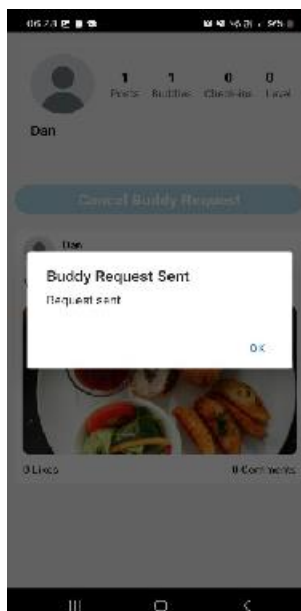
Test case: User profile that was clicked on should be rendered after routing

Result: Success



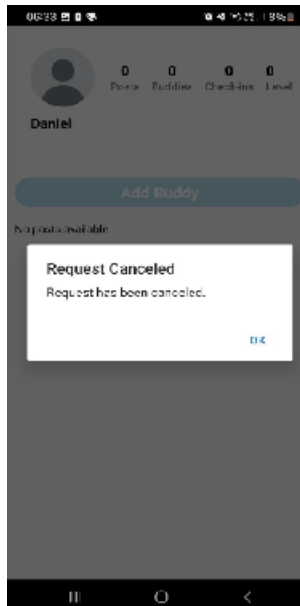
Test case: Upon clicking 'add buddy' button, the data should be committed to database, a message would display 'Buddy request sent' and the 'add buddy' button would change to 'Cancel request'.

Result: Success



Test case: Upon clicking 'Cancel request', the data should delete the buddy request from database, a message should display 'Buddy request cancelled' and the 'Cancel Request' button should change back to 'Add buddy'.

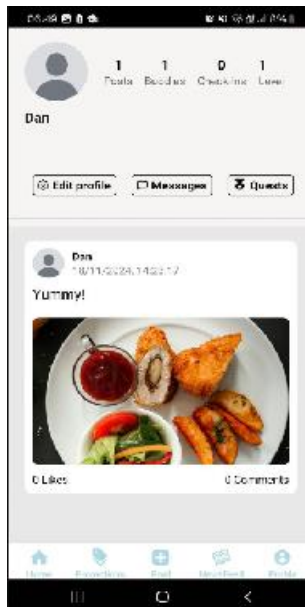
Result: Success



2.3.10 Profile tab

Use case: Profile card and post cards should render

Result: Success



Test case: Quest button should route to Quest Screen when clicked

Result: Success

2.3.11 Quest Screen

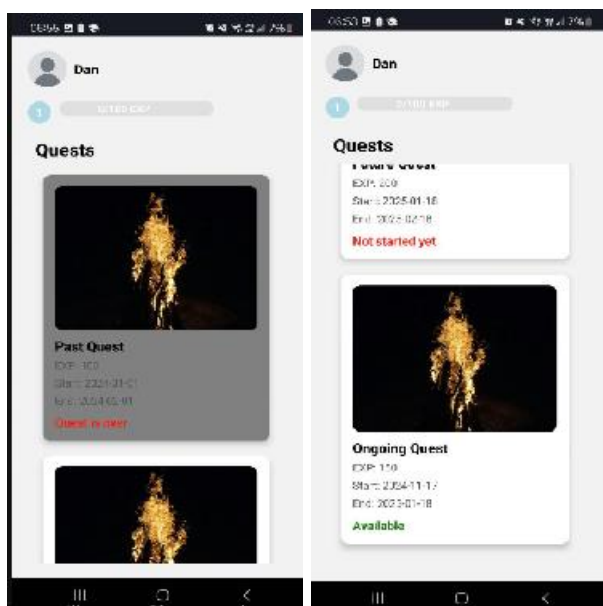
Test case: Quest screen should display user info on quest card and 'no quests' if no quest is available from database.

Result: Success



Test case: Quest screen should display 3 types of quests 'available', 'not started' and 'Quest is over'. Available quests should have a white background with green 'available' text on it. Not started quests should have a white background with red 'Not started Yet' text on it. Quests that are over should have a dark grey background with red 'Quest is over' text on it.

Result: Success



Chapter 3: Project planning

Jira was instrumental in my project planning, serving as the primary tool for organizing and tracking work with efficiency and precision. I structured the project by leveraging Epics, which functioned as high-level objectives or overarching themes that grouped related tasks. These Epics provided a strategic framework, acting as "umbrellas" to encapsulate and align associated work streams.

3.1 Jira

Jira served as the central tool for planning, managing, and tracking the project's progress. Its robust features allowed me to structure the project into logical components, ensuring clear organization and streamlined workflows. Using Jira, I established a hierarchy of work with Epics, User Stories, and associated tasks, enabling efficient delegation, prioritization, and monitoring of progress.

The tool's visual representations, such as Kanban boards and timelines, provided real-time insights into task status and dependencies, fostering better planning and coordination. Additionally, Jira's reporting and analytics capabilities allowed me to measure performance, track key milestones, and identify areas for improvement. By leveraging Jira's comprehensive suite of features, I maintained focus and alignment with project objectives throughout the development lifecycle.

Backlog

MK



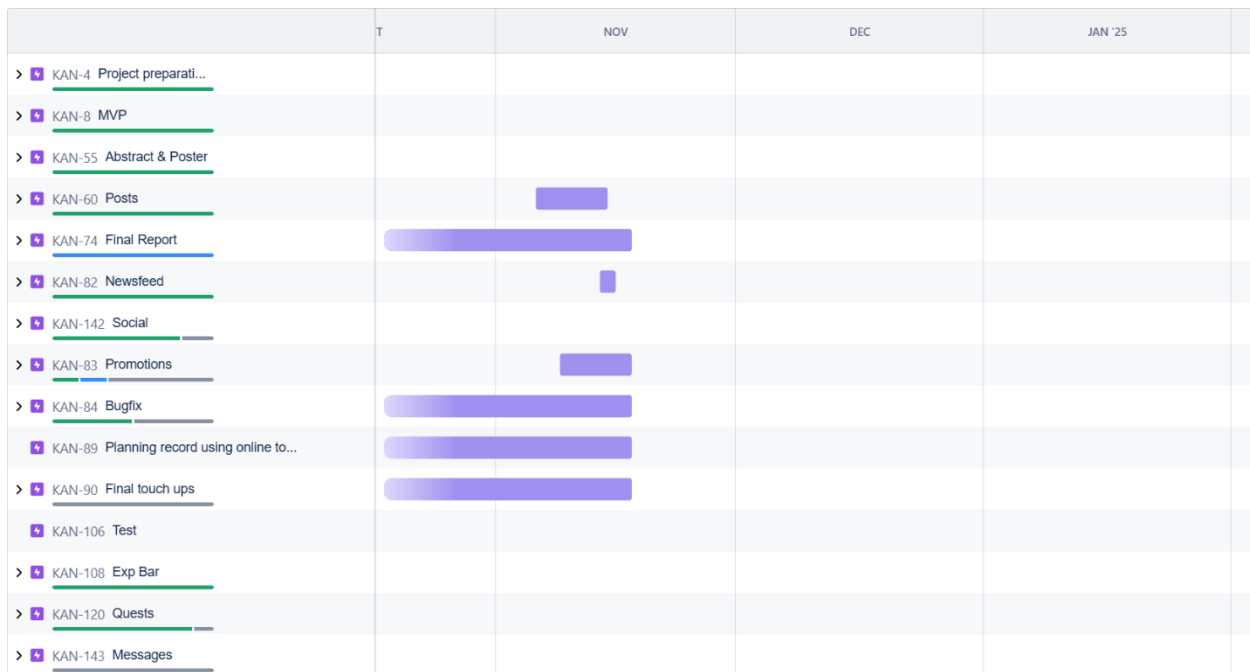
Epic

☐ Board (23 issues)

☒ KAN-66	Find a suitable free to use cloud for image storing	POSTS	DONE
☒ KAN-61	Create cloud account for image uploads	POSTS	DONE
☒ KAN-68	Create and set up bucket for image storing	POSTS	DONE
☒ KAN-62	Create SQL table for user posts	POSTS	DONE
☒ KAN-69	Create comments table in SQL	POSTS	DONE
☒ KAN-70	Create Likes table in SQL	POSTS	DONE
☒ KAN-67	Create model in backend for posts	POSTS	DONE
☒ KAN-72	Create model in backend for comments table	POSTS	DONE
☒ KAN-71	Create model in backend for likes table	POSTS	DONE
☒ KAN-73	Create test routes to test if backend is retrieving the data correctly	POSTS	DONE

3.1.1 Epics

Epics in Jira served as the foundation for organizing high-level objectives of the project. Each Epic represented a major goal or theme and acted as a container for related tasks and deliverables. By grouping related work under Epics, I ensured that the project was strategically aligned and well-structured, allowing for clear prioritization and focus on achieving specific outcomes.



3.1.2 Stories

User Stories were used to break down the high-level objectives defined in Epics into smaller, actionable tasks. Each Story described a specific requirement or task in detail, making it easier to plan, execute, and track progress. Stories also helped assign responsibility and set clear expectations for team members, ensuring that individual contributions were aligned with broader project goals.

⋮	KAN-66	Find a suitable f...	✎	==	MK	DONE ▾
✓	KAN-61	Create cloud account f...		==	MK	DONE ▾
✓	KAN-68	Create and set up buc...		==	MK	DONE ▾
✓	KAN-62	Create SQL table for u...		==	MK	DONE ▾
✓	KAN-69	Create comments tabl...		==	MK	DONE ▾
✓	KAN-70	Create Likes table in S...		==	MK	DONE ▾
⋮	KAN-67	Create model in backe...		==	MK	DONE ▾
✓	KAN-72	Create model in backe...		==	MK	DONE ▾
✓	KAN-71	Create model in backe...		==	MK	DONE ▾
✓	KAN-73	Create test routes to t...		==	MK	DONE ▾

3.1.3 Retrospective

At the end of each Epic, Retrospectives were conducted to review progress and reflect on the project workflow. These sessions provided an opportunity to evaluate what worked well, identify areas for improvement, and address challenges. The insights gathered during these reviews informed adjustments for future work, ensuring continuous improvement and alignment with the project's overall objectives.

▼ Retrospective 	
Retrospective (Project Planning) 	
Retrospective MVP 	
Testing 	
Abstract & Poster 	
Posts 	
Newsfeed 	
Bugfix 	

3.1.4 Did the Methodology Prove Suitable?

The methodology adopted for the project, centered around the use of Jira for Agile-based planning and execution, proved to be highly effective overall. While it streamlined the project's progress and fostered adaptability, some limitations were observed.

Pros

Efficient Organization: The use of Epics and User Stories provided a well-structured framework for organizing tasks.

Iterative Improvement: Regular Retrospectives helped address challenges, and implement improvements after each Epic, ensuring consistent alignment with project goals.

Progress Tracking: The visual dashboards and reports helped monitor progress against milestones, improving transparency and decision-making.

Prioritization: The methodology allowed for flexible prioritization of tasks, helping to focus on delivering high-value outcomes.

Cons

Tool Dependency: Heavy reliance on Jira meant disruptions in its functionality could have negatively impacted the workflow.

Evaluating the Use of Epics Over Sprints: Using Epics instead of Sprints provided a longer and more flexible workflow, allowing for a broader focus on overarching objectives without the constraints of shorter timeframes. This approach was particularly beneficial for managing larger tasks that required sustained effort and strategic alignment. However, the downside of this flexibility was the absence of the structured, weekly iterations characteristic of Sprints, which could have driven faster progress and more frequent deliverables.

3.1.5 Risk Management

Technical risks

Assessment: Moderate

Compatibility issues:

Risk Mitigation: Prioritize compatibility testing on a variety of devices and browsers during the development phase. Use cross-browser testing tools to identify and address compatibility issues early in the development process.

Integration challenges:

Risk Mitigation: Break down integration tasks into smaller, manageable components and conduct thorough testing at each stage of integration. Use standardized APIs and communication protocols to streamline integration efforts.

Performance bottlenecks:

Risk Mitigation: Perform performance testing throughout the development lifecycle to identify potential bottlenecks and optimize code, database queries, and network interactions. Implement caching mechanisms and scaling strategies to handle increased user load.

Security risks

Assessment: High

Data breaches:

Risk Mitigation: Implement robust security measures such as encryption, access controls, and data masking to protect sensitive data from unauthorized access. Conduct regular security audits and penetration testing to identify and remediate

vulnerabilities.

Authentication flaws:

Risk Mitigation: Use industry-standard authentication mechanisms such as multi-factor authentication (MFA) and OAuth for user authentication. Regularly update authentication libraries and frameworks to address known vulnerabilities.

Operational risks

Assessment: Moderate

Data loss:

Risk Mitigation: Implement data backup and recovery procedures to prevent data loss in the event of system failures or disasters. Use redundant storage systems and automated backup solutions to ensure data integrity and availability.

Server downtime:

Risk Mitigation: Implement high availability and fault-tolerant architectures such as load balancing, failover clustering, and redundant infrastructure components. Monitor server health and performance in real-time and proactively address issues to minimize downtime.

Market risks

Assessment: High

Changing user preferences:

Risk Mitigation: Stay informed about market trends and user preferences through market research, surveys, and user feedback. Build flexibility into the application design to adapt to changing user needs and preferences.

New & untested:

Risk Mitigation: I would launch a beta application first to be able to anticipate market response and adapt accordingly.

Competitive pressure:

Risk Mitigation: Differentiate the application by focusing on unique features, user experience, and value proposition. Monitor competitor activities and market dynamics to identify opportunities and threats.

Regulatory risks

Assessment: High

Compliance requirements:

Risk Mitigation: Stay informed about relevant regulations and compliance

requirements in the target market. Engage legal experts to ensure compliance with data protection regulations and consumer privacy laws.

Resource risks

Assessment: Moderate

Inadequate experience:

Risk Mitigation: Do more research and practice on the tools that will be used in the project

Resource constraints:

Risk Mitigation: Prioritize tasks and allocate resources based on project priorities and critical path activities.

[Risk Management](#) 



Chapter 4: Conclusions

This chapter provides an overview of the project's key results, emphasizing accomplishments, areas needing improvement, challenges faced, and prospects for future enhancements. It also highlights the insights and skills developed during the course of the project.

4.2 Bugs & Unimplemented functions

While there were many successes in this project, not everything went as planned, and there were several challenges that remained unresolved by the end of the development cycle.

4.2.1 Bugs

Currently, there is an issue with connecting to Supabase for the upload of images and videos, as well as retrieving their corresponding URLs. I am actively investigating the root cause, which may be related to factors such as bucket policies or configuration settings. Additionally, there is a bug related to password encryption, which is being addressed. It is expected that both issues will be resolved before the final presentation.

4.2.2 Unimplemented functions

Due to the ongoing issue with connecting to Supabase, users are currently unable to create posts, which has led to a significant bottleneck in the project's functionality. Several key features are dependent on users being able to post content. For example, users cannot complete quests, which are integral to earning experience points, leveling up, and advancing through tiers. This limitation directly impacts the user progression system, where tier-based rewards and incentives are crucial.

Additionally, the inability to connect to Supabase affects the Promotions page, as promotions are tied to user tiers and levels. Without the ability to post, users are unable to progress through tiers, thus rendering the promotions feature non-functional at this time.

Furthermore, the lack of post creation functionality also hampers the review and comment system. Reviews and comments are intended to include media such as photos and videos; without the connection to Supabase, these features are restricted to text-based content only, reducing the overall user experience and engagement.

4.3 Future Work

There is significant potential for improvement and expansion of the platform, both in terms of features and scalability. Several areas can be further developed to enhance user experience, streamline operations, and increase the platform's capability to scale with growth.

4.3.1 Features

One area for future enhancement is the addition of a restaurant registration and login system. This would enable restaurant owners to manage their own promotions by editing and posting them directly on the platform, rather than relying on manual updates to the database. This feature would not only improve operational efficiency but also allow restaurants to have greater control over their offerings.

Another potential feature is the introduction of a loyalty points system, where users can earn points by engaging with restaurant promotions, leaving reviews, or completing specific actions. These points could be redeemed for exclusive deals or discounts, incentivizing customer engagement and fostering long-term loyalty.

Integrating the Google Maps API would also greatly improve the user experience. By allowing restaurants to pinpoint their exact locations on the map, users would be able to easily find directions and navigate to their desired destinations. This addition would enhance the platform's functionality, making it more user-friendly, especially for those looking for nearby dining options.

Additionally, incorporating a more robust review system that includes multimedia content, such as photos and videos, would allow users to provide richer, more detailed feedback about their experiences. This could further engage users and offer greater value for other diners looking to make informed decisions.

4.3.2 Scalability

As the platform continues to grow, the scalability of both the backend and external services will become increasingly important. To accommodate a larger user base and an expanding database, the free-tier plans for services like Supabase and Google Maps API may need to be upgraded. Moving to a higher-tier plan would provide additional storage capacity, more API calls, and better overall performance. This would ensure that the platform remains responsive and capable of handling increased demand, particularly as the volume of posts, promotions, and interactions grows.

Further scalability considerations could include exploring cloud-based solutions for data storage, improving load balancing, and enhancing the infrastructure to support increased traffic, especially during peak usage periods.

By addressing these areas, the platform could not only meet the current demands of users but also position itself for sustained growth and success in the future.

4.4 Learning Outcomes

Throughout the course of this project, I gained valuable insights into various technical and non-technical aspects of software development. This experience deepened my understanding of the tools, frameworks, and methodologies used to build a functional and scalable application, while also enhancing my problem-solving and project management skills.

One of the most significant learning outcomes was my proficiency in using modern web development technologies, particularly Expo for mobile application development, and Supabase for backend services. Working with Supabase taught me how to implement storage solutions for media like images and videos

Additionally, I gained experience with SQL, Flask, and Elasticsearch, which were crucial for the backend of the application. Working with SQL deepened my understanding of relational database design and querying, while Flask allowed me to efficiently build and manage APIs to interact with the frontend. Elasticsearch taught me how to implement full-text search and optimize data retrieval, enhancing the overall performance of the application. This combination of technologies expanded my skill set.

Furthermore, the project provided me with a deeper appreciation for the role of agile methodologies, particularly when using tools like Jira for project management. By utilizing Epics, User Stories, and Retrospectives, I was able to structure the development process efficiently, breaking down complex tasks into manageable pieces and maintaining a clear overview of progress. The iterative approach encouraged continuous reflection and improvement, ensuring that I stayed aligned with project objectives throughout the development lifecycle.

Another significant learning outcome was the ability to collaborate and manage time effectively. This project required balancing multiple tasks and priorities, such as backend setup, frontend development, and continuous testing. Developing a strong understanding of task prioritization, maintaining focus, and adjusting plans in response to setbacks became essential for ensuring steady progress.

In summary, this project not only allowed me to develop and apply technical skills but also provided opportunities to strengthen my project management abilities, problem-solving mindset, and adaptability. The learning outcomes gained from this experience have laid a solid foundation for my future endeavors in software development.

References

- [1] A. J. Kucharczuk, T. L. Oliver, and E. B. Dowdell, "Social media's influence on adolescents' food choices: A mixed studies systematic literature review," *Appetite*, vol. 168, p. 105765, Jan. 2022. [Online]. Available: <https://doi.org/10.1016/j.appet.2021.105765>.
- [2] V. Patwardhan, J. Mallya, K. S., and D. K. Welcomgroup, "Influence of social media on young adults' food consumption behavior: scale development," *Cogent Social Sciences*, vol. 10, no. 1, p. 2391016, Aug. 2024. [Online]. Available: <https://doi.org/10.1080/23311886.2024.2391016>.
- [3] J. Kim, J. Jun, and L. (R.) Tang, "How well does advertising work on restaurant performance? A dynamic and quadratic approach," *International Journal of Hospitality Management*, vol. 77, pp. 153-164, 2019. [Online]. Available: https://e-tarjome.com/storage/panel/fileuploads/2019-09-24/1569329430_E13580-e-tarjome.pdf.
- [4] EHL Faculty, "Online customer reviews: Their impact on restaurants," *EHL Insights*, Sep. 30, 2018. [Online]. Available: <https://www.ehl.edu/en/insights/online-customer-reviews-their-impact-on-restaurants>.
- [5] A. M. E. S. Abdo, "The interrelationship between niche marketing and Competitiveness," *Advertising Dep., Faculty of Applied Arts, Damietta University*, Mar. 2021. [Online]. Available: https://idj.journals.ekb.eg/article_234783_2300db0e4dbc28ee05d42c3432bc37f6.pdf.
- [6] [7] S. Wareebor, C. Suttikun, and P. Mahasuweerachai, "Exploring the influence of online restaurant promotions on consumer behavioral intentions,"

Journal of Hospitality and Tourism Insights, vol. ahead-of-print, no. ahead-of-print, 2024. [Online]. Available: <https://doi.org/10.1108/JHTI-05-2024-0511>.

[8] H. Hafid and I. Kucukköse, "Discovering a need for marketing services in the small restaurant industry," M.S. thesis, Business Administration, Master of Science Thesis, 30 ECTS, Spring 2022, Supervisor: J. Trischler. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1689008/FULLTEXT01.pdf>.